

Application 1 : Gestion de Produits (comme ton exemple)

Structure :

```
src/  
├─ app/  
│   └─ store.js  
├─ features/  
│   └─ products/  
│       ├── ProductSlice.js  
│       ├── ProductList.js  
│       └─ Product.test.js  
└─ index.js
```

ProductSlice.js : (redux toolkit)

```
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";  
  
// Async thunk simule fetch API  
export const fetchProducts = createAsyncThunk(  
  "products/fetchProducts",  
  async () => {  
    return [  
      { id: 1, name: "Laptop", category: "Electronics" },  
      { id: 2, name: "Phone", category: "Electronics" },  
    ];  
  }  
);  
  
const initialState = {  
  products: [],  
  search: "",  
  category: "",  
  status: "idle",  
};  
  
const productSlice = createSlice({  
  name: "products",  
  initialState,  
  reducers: {  
    searchByName: (state, action) => { state.search = action.payload },  
    filterByCategory: (state, action) => { state.category = action.payload },  
  },  
  extraReducers: (builder) => {  
    builder  
      .addCase(fetchProducts.fulfilled, (state, action) => {  
        state.products = action.payload;  
        state.status = "success";  
      })  
  };  
});
```

```
export const { searchByName, filterByCategory } = productSlice.actions;
export default productSlice.reducer;
```

Product.test.js :

```
import reducer, { searchByName, filterByCategory, fetchProducts } from "../ProductSlice";

describe("productSlice reducers", () => {
  const initialState = { products: [], search: "", category: "", status: "idle" };

  it("should handle searchByName", () => {
    const state = reducer(initialState, searchByName("Laptop"));
    expect(state.search).toBe("Laptop");
  });

  it("should handle filterByCategory", () => {
    const state = reducer(initialState, filterByCategory("Electronics"));
    expect(state.category).toBe("Electronics");
  });
});

describe("productSlice extraReducers", () => {
  it("should handle fetchProducts.fulfilled", () => {
    const action = { type: fetchProducts.fulfilled.type, payload: [{ id: 1, name: "Laptop", category: "Electronics" }] };
    const state = reducer(undefined, action);
    expect(state.products).toHaveLength(1);
    expect(state.status).toBe("success");
  });
});
```

Application 2 : Todo List avec Redux Toolkit + Tests

Slice (TodoSlice.js)

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = { todos: [] };

const todoSlice = createSlice({
  name: "todos",
  initialState,
  reducers: {
    addTodo: (state, action) => { state.todos.push(action.payload) },
    removeTodo: (state, action) => { state.todos = state.todos.filter(todo => todo.id !== action.payload) }
  }
});

export const { addTodo, removeTodo } = todoSlice.actions;
export default todoSlice.reducer;
```

Test (TodoSlice.test.js)

```
import reducer, { addToDo, removeToDo } from "./TodoSlice";

describe("todoSlice reducers", () => {
  it("should add a todo", () => {
    const state = reducer({ todos: [] }, addToDo({ id: 1, text: "Learn Jest" }));
    expect(state.todos).toHaveLength(1);
    expect(state.todos[0].text).toBe("Learn Jest");
  });

  it("should remove a todo", () => {
    const initial = { todos: [{ id: 1, text: "Learn Jest" }] };
    const state = reducer(initial, removeToDo(1));
    expect(state.todos).toHaveLength(0);
  });
});
```

Application 3 : Counter avec Redux Toolkit + React Component + Test

CounterSlice.js

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = { value: 0 };

const counterSlice = createSlice({
  name: "counter",
  initialState,
  reducers: {
    increment: state => { state.value += 1 },
    decrement: state => { state.value -= 1 }
  }
});

export const { increment, decrement } = counterSlice.actions;
export default counterSlice.reducer;
```

Counter.js (React Component)

```
import React from "react";
import { useSelector, useDispatch } from "react-redux";
import { increment, decrement } from "./CounterSlice";

export default function Counter() {
  const count = useSelector(state => state.counter.value);
  const dispatch = useDispatch();

  return (
    <div>
      <h1>Count: {count}</h1>
    </div>
  );
}
```

```

    <button onClick={() => dispatch(increment())}>+</button>
    <button onClick={() => dispatch(decrement())}>-</button>
  </div>
);
}

```

Counter.test.js (RTL + Jest)

```

import { render, screen, fireEvent } from "@testing-library/react";
import { Provider } from "react-redux";
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "../CounterSlice";
import Counter from "../Counter";

test("counter increments and decrements", () => {
  const store = configureStore({ reducer: { counter: counterReducer } });
  render(
    <Provider store={store}>
      <Counter />
    </Provider>
  );

  const count = screen.getByText(/Count:/i);
  expect(count).toHaveTextContent("0");

  fireEvent.click(screen.getByText("+"));
  expect(count).toHaveTextContent("1");

  fireEvent.click(screen.getByText("-"));
  expect(count).toHaveTextContent("0");
});

```

Application 4: Todo List Avancée

Structure

```

src/
├── app/store.js
├── features/todos/ToDoSlice.js
├── features/todos/ToDoList.js
├── features/todos/ToDo.test.js
└── index.js

```

ToDoSlice.js

```

import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  todos: [],
  filter: "all" // all | completed | incomplete
};

```

```

const todoSlice = createSlice({
  name: "todos",
  initialState,
  reducers: {
    addTodo: (state, action) => state.todos.push(action.payload),
    removeTodo: (state, action) => {
      state.todos = state.todos.filter(todo => todo.id !== action.payload);
    },
    toggleTodo: (state, action) => {
      const todo = state.todos.find(todo => todo.id === action.payload);
      if (todo) todo.completed = !todo.completed;
    },
    setFilter: (state, action) => {
      state.filter = action.payload;
    }
  }
});

export const { addTodo, removeTodo, toggleTodo, setFilter } = todoSlice.actions;
export default todoSlice.reducer;

```

Todo.test.js

```

import reducer, { addTodo, removeTodo, toggleTodo, setFilter } from "../TodoSlice";

describe("TodoSlice reducers", () => {
  it("should add a todo", () => {
    const state = reducer({ todos: [], filter: "all" }, addTodo({ id: 1, text: "Learn Redux", completed: false }));
    expect(state.todos).toHaveLength(1);
    expect(state.todos[0].text).toBe("Learn Redux");
  });

  it("should remove a todo", () => {
    const initial = { todos: [{ id: 1, text: "Learn Redux", completed: false }], filter: "all" };
    const state = reducer(initial, removeTodo(1));
    expect(state.todos).toHaveLength(0);
  });

  it("should toggle a todo", () => {
    const initial = { todos: [{ id: 1, text: "Learn Redux", completed: false }], filter: "all" };
    const state = reducer(initial, toggleTodo(1));
    expect(state.todos[0].completed).toBe(true);
  });

  it("should set filter", () => {
    const state = reducer({ todos: [], filter: "all" }, setFilter("completed"));
    expect(state.filter).toBe("completed");
  });
});

```

Application 5: Counter Avancé avec composants et thunk async

Structure

```
src/  
├─ features/counter/CounterSlice.js  
├─ features/counter/Counter.js  
└─ features/counter/Counter.test.js
```

CounterSlice.js

```
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";  
  
export const incrementAsync = createAsyncThunk(  
  "counter/incrementAsync",  
  async () => {  
    return new Promise(resolve => setTimeout(() => resolve(1), 500));  
  }  
);  
  
const initialState = { value: 0 };  
  
const counterSlice = createSlice({  
  name: "counter",  
  initialState,  
  reducers: {  
    increment: state => { state.value += 1 },  
    decrement: state => { state.value -= 1 },  
    reset: state => { state.value = 0 }  
  },  
  extraReducers: builder => {  
    builder.addCase(incrementAsync.fulfilled, (state, action) => {  
      state.value += action.payload;  
    });  
  }  
});  
  
export const { increment, decrement, reset } = counterSlice.actions;  
export default counterSlice.reducer;
```

Counter.js

```
import React from "react";  
import { useSelector, useDispatch } from "react-redux";  
import { increment, decrement, reset, incrementAsync } from "../CounterSlice";  
  
export default function Counter() {  
  const count = useSelector(state => state.counter.value);  
  const dispatch = useDispatch();  
  
  return (  
    <div>  
      <h1>Count: {count}</h1>  
      <button onClick={() => dispatch(increment())}>+</button>  
      <button onClick={() => dispatch(decrement())}>-</button>  
    </div>  
  );  
}
```

```

    <button onClick={() => dispatch(reset())}>Reset</button>
    <button onClick={() => dispatch(incrementAsync())}>+ Async</button>
  </div>
);
}

```

Counter.test.js

```

import { render, screen, fireEvent } from "@testing-library/react";
import { Provider } from "react-redux";
import { configureStore } from "@reduxjs/toolkit";
import counterReducer, { incrementAsync } from "./CounterSlice";
import Counter from "./Counter";

jest.useFakeTimers();

test("counter increment, decrement, reset, async", async () => {
  const store = configureStore({ reducer: { counter: counterReducer } });
  render(<Provider store={store}><Counter /></Provider>);

  const count = screen.getByText(/Count:/i);
  const incrementBtn = screen.getByText("+");
  const decrementBtn = screen.getByText("-");
  const resetBtn = screen.getByText("Reset");
  const asyncBtn = screen.getByText("+ Async");

  fireEvent.click(incrementBtn);
  expect(count).toHaveTextContent("1");

  fireEvent.click(decrementBtn);
  expect(count).toHaveTextContent("0");

  fireEvent.click(resetBtn);
  expect(count).toHaveTextContent("0");

  fireEvent.click(asyncBtn);
  jest.runAllTimers();
  await Promise.resolve(); // attendre la promesse
  expect(count).toHaveTextContent("1");
});

```

Application 6 : Gestion de Produits avec thunk fetch simulé

Structure

```

src/
├── features/products/ProductSlice.js
├── features/products/ProductList.js
└── features/products/Product.test.js

```

ProductSlice.js

```

import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";

```

```
export const fetchProducts = createAsyncThunk(
  "products/fetchProducts",
  async () => {
    return [
      { id: 1, name: "Laptop", category: "Electronics" },
      { id: 2, name: "Phone", category: "Electronics" }
    ];
  }
);

const initialState = { products: [], status: "idle" };

const productSlice = createSlice({
  name: "products",
  initialState,
  reducers: {},
  extraReducers: builder => {
    builder.addCase(fetchProducts.fulfilled, (state, action) => {
      state.products = action.payload;
      state.status = "success";
    });
  }
});

export default productSlice.reducer;
```

Product.test.js

```
import reducer, { fetchProducts } from "../ProductSlice";

describe("ProductSlice extraReducers", () => {
  it("should handle fetchProducts.fulfilled", () => {
    const action = { type: fetchProducts.fulfilled.type, payload: [{ id: 1, name: "Laptop" }] };
    const state = reducer(undefined, action);
    expect(state.products).toHaveLength(1);
    expect(state.status).toBe("success");
  });
});
```